॥वसुधैव कुटुम्बकम्॥

Symbiosis Institute of Technology, Pune

Faculty of Engineering

CSE - Academic Year 2026-27

Java Programming – Lab (Batch 2025-29)

Lab Assignment No:- 26

Name of Student	Abobakr Mohamed Elameen
PRN No.	25070122015
Batch	2025-29
Class	Programming with JAVA
Academic Year & Semester	2nd Year – Sem III
Date of Submission	03/10/2026
Title of Assignment	Write a Java program to create and execute multiple threads using Runnable interface.
Practice questions	1. Create a program using Runnable to run two tasks simultaneously.

```java
package JAVA_EXP_26;

class NumberTask implements Runnable {

    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Number Task   : " + i);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println("Number Task interrupted");
            }
        }
    }
}

class LetterTask implements Runnable {

    public void run() {
        for (char ch = 'A'; ch <= 'E'; ch++) {
```

```
            System.out.println("Alphabet Task : " + ch);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println("Alphabet Task interrupted");
            }
        }
    }
}

public class RunnableTasks {

    public static void main(String[] args) throws
InterruptedException {
        Thread t1 = new Thread(new NumberTask(), "Number-Thread");
        Thread t2 = new Thread(new LetterTask(), "Alphabet-
Thread");

        System.out.println(t1.getName() + " state: " +
t1.getState());
        System.out.println(t2.getName() + " state: " +
t2.getState());

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println(t1.getName() + " state: " +
t1.getState());
        System.out.println(t2.getName() + " state: " +
t2.getState());
        System.out.println("Both tasks have completed.");
    }
}
```

2. Create a Java program using the Runnable interface where two tasks run simultaneously: one thread performs deposit operations, and another performs withdrawal operations, both displaying results concurrently.

```
package JAVA_EXP_26;
class Account {
    private int balance;

    Account(int balance) {
        this.balance = balance;
```

```
    }

    synchronized void deposit(int amount) {
        balance += amount;
        System.out.println(Thread.currentThread().getName() + "
deposited " + amount + " | Balance: " + balance);
    }

    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
            System.out.println(Thread.currentThread().getName() +
" withdrew " + amount + " | Balance: " + balance);
        } else {
            System.out.println(Thread.currentThread().getName() +
" failed to withdraw " + amount + " | Insufficient balance: " +
balance);
        }
    }

    synchronized int getBalance() {
        return balance;
    }
}

class DepositTask implements Runnable {
    private Account account;

    DepositTask(Account account) {
        this.account = account;
    }

    public void run() {
        int[] amounts = {1000, 2000, 1500, 500};
        for (int amount : amounts) {
            account.deposit(amount);
            try {
                Thread.sleep(400);
            } catch (InterruptedException e) {
                System.out.println("Deposit task interrupted");
            }
        }
    }
}

class WithdrawTask implements Runnable {
    private Account account;

    WithdrawTask(Account account) {
```

```
        this.account = account;
    }

    public void run() {
        int[] amounts = {800, 3000, 1200, 6000};
        for (int amount : amounts) {
            account.withdraw(amount);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println("Withdraw task interrupted");
            }
        }
    }
}

public class RunnableBank {

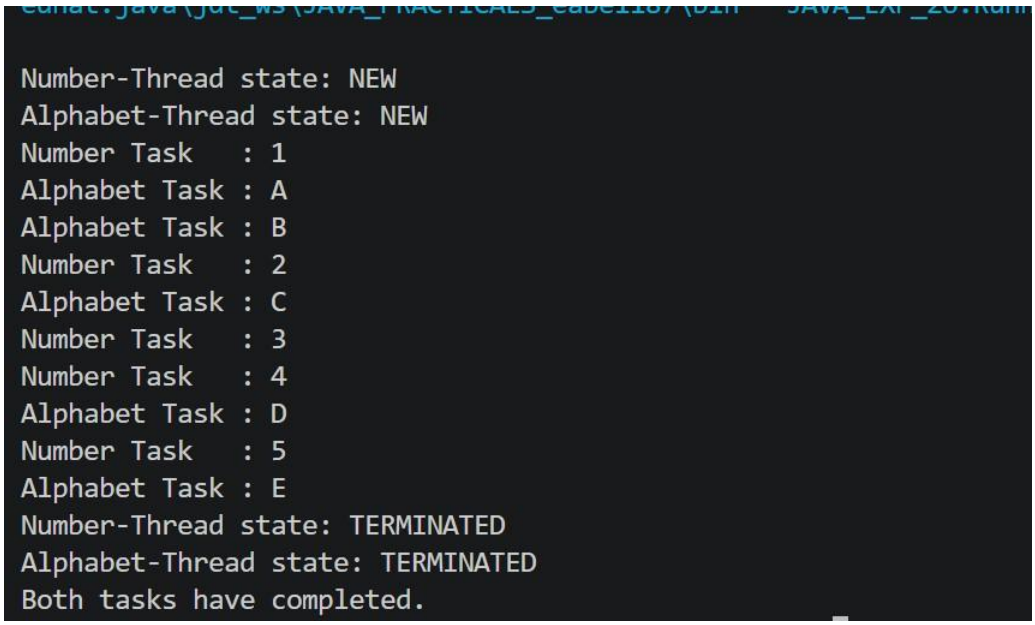
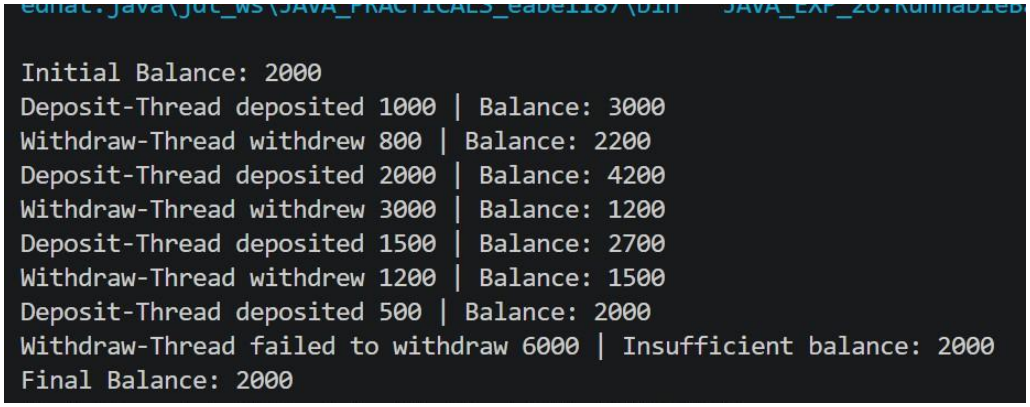
    public static void main(String[] args) throws
InterruptedException {
        Account account = new Account(2000);
        System.out.println("Initial Balance: " +
account.getBalance());

        Thread depositThread = new Thread(new
DepositTask(account), "Deposit-Thread");
        Thread withdrawThread = new Thread(new
WithdrawTask(account), "Withdraw-Thread");

        depositThread.start();
        withdrawThread.start();

        depositThread.join();
        withdrawThread.join();

        System.out.println("Final Balance: " +
account.getBalance());
    }
}
```

Output	1)  2) 
Conclusion	Thus, we have successfully created and executed multiple threads in Java using the Runnable interface. The programs show tasks running concurrently, including deposit and withdrawal operations on a shared account, and how Runnable keeps task logic separate from thread execution.